The Journal of Systems and Software 237 (2026) 112849

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

# BEWT: Extended Benchmarking for End-to-End Web Testing ⋆

Dario Olianas, Maurizio Leotta *, Filippo Ricca

*Dipartimento di Informatica, Bioingegneria, Robotica e Ingegneria dei Sistemi (DIBRIS), Università di Genova, Genova, Italy*

ARTICLE INFO

Editor: Prof W. Eric Wong

*Keywords:*
End-to-end testing
Testing dataset
Web testing
Selenium

ABSTRACT

Web applications are essential in modern society and require thorough testing to ensure reliability and dependability. End-to-End (E2E) testing is key to evaluating both front-end and back-end components of complex web applications.

Recent research has focused on improving E2E test suites by reducing maintenance costs, mitigating flakiness, and enhancing test resilience. Yet, a major challenge remains the lack of a publicly available benchmark for comparing techniques. Developing test suites for research is complex and prone to bias, highlighting the need for a shared benchmark to facilitate comparisons and experimental validation.

We address this gap by providing a set of test suites designed as a benchmark for E2E testing studies. They come with scripts and automated installers for seamless deployment of the application under test in Docker containers, enhancing their usability. Our benchmark consists of 36 Java Selenium WebDriver-based E2E test suites for 8 different web applications. Each application includes multiple test suites with varying characteristics, such as the use of the Page Object pattern and advanced waiting mechanisms. Additionally, for 4 out of the 8 web applications, we provide test suites for two different versions, enabling studies on test suite evolution. Our benchmark includes 1166 test scripts, featuring 492 Page Objects and 8284 locators. To manage asynchronous behavior and reduce flakiness, the test suites incorporate 718 thread sleeps and 815 explicit waits. With a total of 51838 lines of code (LOC) this dataset can potentially be used as a resource, for example, to evaluate test automation strategies, study test suite evolution, or explore flakiness mitigation techniques.

## 1. Introduction

Web applications play a central role in modern society, supporting a wide range of activities. Ensuring their reliability is critical to reduce potential errors. One widely used approach for verifying application correctness is End-to-End (E2E) testing, which evaluates the system as a whole, encompassing both the front-end and back-end components. Due to the inherent complexity and distribution of web applications across multiple systems, E2E testing replicates realistic user interactions, navigating the GUI elements and exercising the application's workflows in a manner similar to human testers.

Over time, numerous frameworks and tools have been proposed to support the implementation and execution of E2E tests for web applications. Test scripts can be created following either the Capture & Replay approach or Script-Based techniques (Leotta et al., 2016a). Moreover, various types of locators can be employed to identify and manipulate web page elements (Nass et al., 2023; Leotta et al., 2021). Consequently, a broad range of research contributions have introduced algorithms, methodologies, and tools aimed at improving E2E test suites. Key objectives include reducing the initial cost of test script creation (Leotta et al., 2022b), lowering maintenance overhead (Stocco et al., 2017), enhancing the robustness of test scripts to changes in the web application (Leotta et al., 2020), and mitigating test flakiness (Olianas et al., 2021). Whenever a new approach, tool, or algorithm is proposed, it is essential to empirically validate its effectiveness, providing evidence of improvements in the target aspects of the test suite. For instance, when introducing a novel algorithm for generating locators to identify GUI elements during test execution, it is crucial to demonstrate that these locators are more reliable than those produced by existing state-of-the-art tools, especially considering typical web application evolution.

A major challenge in the current research landscape is the absence of a publicly available benchmark that enables systematic comparison of different approaches. Researchers typically need to select a set of test suites to assess the effectiveness of a new technique, but such resources are limited. Constructing new test suites specifically for an experiment can also introduce biases that may compromise the validity of the study.

This motivates the need for a shared benchmark dataset. The adoption of such a benchmark allows researchers to compare new techniques

⋆ Editor's note: Open Science material was validated by the Journal of Systems and Software Open Science Board.

* Corresponding author.
*E-mail addresses:* dario.olianas@dibris.unige.it (D. Olianas), maurizio.leotta@unige.it (M. Leotta), filippo.ricca@unige.it (F. Ricca).

https://doi.org/10.1016/j.jss.2026.112849
Received 30 November 2025; Received in revised form 16 February 2026; Accepted 6 March 2026
Available online 11 March 2026
0164-1212/© 2026 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

against existing ones without repeatedly executing previously developed tools. Once validated on the benchmark, results can be directly compared with future approaches, streamlining empirical evaluation.

In this work, we present BEWT (Benchmark for E2E Web Testing), an extended collection of test suites designed as a benchmark for studies in E2E testing. The suites were carefully curated and refined to provide comprehensive features suitable for evaluating novel techniques in this domain. Each test suite is accompanied by a Docker image or Docker compose YAML to easily deploy the corresponding web application in a Docker container, enabling fast, reliable, and reproducible deployment.

BEWT provides a controlled and reproducible setting to systematically assess and compare new E2E web testing approaches and tools across the eight benchmark applications, by using uniformly engineered test suites and systematically derived variants under a standardized execution setup. By enabling rigorous evaluation, the benchmark can contribute to research on safe and dependable web systems, helping reduce the likelihood of undesired states and failures. In this way, BEWT serves as a solid foundation for the development and assessment of automated tools and strategies aimed at enhancing the reliability, maintainability, and overall dependability of web applications, directly aligning with software dependability objectives in critical and complex web systems.

The contributions of this extended work, which builds upon our previous conference paper (Olianas et al., 2026), are as follows:

1. the benchmark has been significantly expanded to allow more extensive studies and experiments. In particular, three new kinds of test suites have been added to the three original Gherkin, Base, and Base+ versions. The total number of implemented test scripts has increased from 389 to 1166 that implement in different ways the 258 test cases defined in Gherkin. The benchmark is now composed by the following test suites versions:
   - **Gherkin version:** the functional specification of each test case is provided using the Gherkin language (a structured natural DSL), which describes in detail every action to be performed on the web application GUI and defines the assertions to be evaluated.
   - **Baseline version (Base):** the base version from which all the other versions derive. Java, Selenium-based test suites, adopting the Page Object design pattern and implicit waits as waiting strategy, supplemented by thread sleeps where required.
   - **Baseline evolution version (Base+):** for 4 applications out of 8, we developed a second version of the test suite for a more recent version of the application under test.
   - **Explicit Wait version (Exp-Wait):** this version employs explicit waits as waiting strategy, instead of the implicit waits used by all the other versions.
   - **No page objects version (No-PO):** this version does not employ the Page Object design pattern.
   - **Full XPath version (Full-XPath):** this version uses only absolute XPaths as locators for web elements.
2. the benchmark description has been enriched with detailed explanations of the newly added types of test suites.
3. the discussion of potential research uses of the benchmark has been greatly expanded, now including detailed descriptions of possible experiments that the benchmark can support.
4. the quality of the test suite code has been improved. All test suites were thoroughly checked for flakiness, and every issue that could reasonably be identified was addressed. However, some flakiness still affects the test suites, particularly in the explicit-wait (Exp-Wait) versions this is intentional, as these versions are meant to support studies focused on flakiness reduction. Moreover, due to the inherently non-deterministic and context-dependent nature of flakiness, residual issues may still occur. Nonetheless, the test suites are now significantly more stable than in the original release.

*BEWT at-a-glance.* BEWT is a curated benchmark for *end-to-end (E2E) web testing* research and tool evaluation. It provides:

- **Systems under test:** 8 open-source web applications, selected to be realistic and reusable.
- **Test assets:** Gherkin specifications (as human-readable test intent) and executable Java/Selenium WebDriver test suites.
- **Systematic suite variants:** multiple engineered variants (e.g., Page Objects vs. no-PO, explicit-wait vs. sleep-based waiting, alternative locator styles) to enable controlled comparisons.
- **Evolution support:** "Base+" suites for newer releases of selected applications to study test evolution over time.
- **Reproducibility:** containerized deployment and scripts to automate environment setup and test execution, enabling independent verification and reuse.

Overall, BEWT goes beyond aggregating existing tests by providing *uniformly engineered*, systematically derived variants and replication support that make cross-technique comparisons feasible.

*Reader's guide.* The paper is organized to support both readers who want to *use* the benchmark and readers interested in *how it was built*. Section 2 provides background on end-to-end (E2E) web testing and may be skipped by readers who are already familiar with E2E testing for web applications and Selenium WebDriver. Section 3 outlines the main research areas enabled by BEWT and presents *illustrative evaluation scenarios* (these are intended as guidance for future studies and can be skimmed on a first read). Section 4 details benchmark construction, selection criteria, and stabilization/reproducibility procedures. Section 5 describes the released test suites and variants and provides the main quantitative characterization of the artifact. Section 6 positions BEWT with respect to related datasets/benchmarks, and Section 7 concludes the paper.

## 2. End-to-End (E2E) web testing

End-to-End (E2E) testing of web applications is a type of black-box testing based on structured test scenarios (Leotta et al., 2016a). These scenarios describe sequences of actions that mimic real user interactions or procedures typically performed during manual testing. For instance, a login scenario may involve entering a username, typing a password, clicking the login button, and checking that authentication was successful.

From a single scenario, multiple test cases can be derived by defining concrete input values (e.g., username = "John.Doe", password = "1234") and specifying the expected outcomes as assertions. The execution of these test cases can then be automated through test scripts created using various approaches (Leotta et al., 2016a, 2024a; Berner et al., 2005). The choice of development method often depends on the technology stack, the available tools (e.g., Selenium WebDriver, Selenium IDE), and the programming expertise of the testers (Leotta et al., 2013).

Test scripts are usually developed following one of three main approaches: Script-Based Testing (SBT), Capture-and-Replay Testing (CRT), or the more recent NLP-based Web Testing (NLT).

*Script-Based Testing* (**SBT**) involves writing test scripts in a programming language such as Java, Python, or Ruby, while leveraging dedicated libraries that provide commands to control browser behavior.

*Capture-and-Replay Testing* (**CRT**) starts with a manual execution where the tester interacts with the application while a tool (e.g., Selenium IDE) records all actions, interactions with web elements, and keystrokes. The recorded session is then automatically converted into test scripts, which can be replayed for automated testing. Recordings may also include assertions, usually appended to verify results after the replay. These scripts can often be modified to adjust inputs and assertions. Depending on the tool, scripts may be maintained in a proprietary language (e.g., Selenese for Selenium IDE) or exported to a standard programming language like Java or Python.

*NLP-based Web Testing* (**NLT**) employs natural language processing techniques, allowing testers to write test cases in plain language. These

```
Feature: User management
Scenario: Creates a new user
    Given the user is on the home page
    When the user clicks the "Log in" link
        And enters "admin" in the "Username" field
        And enters "Password001" in the "Password" field
        And clicks the "Log in" button
        And clicks the "Special pages" link
        And clicks the "Create account" link
        And enters "User001" in the "Username" field
        And enters "Password001" in the "Password" field
        And enters "Password001" in the "Confirm Password" field
        And enters "Real Name 001" in the "Real Name" field
        And clicks the "Create" button
    Then "The user account for User001 (talk) has been created." is displayed

    Given the previous assertion passed
    Then the user clicks the "Log out" link
```

**Fig. 1.** Example of test case specified with Gherkin.

textual descriptions are then automatically translated into executable test scripts by the tool (Leotta et al., 2024a). NLP, as a subset of artificial intelligence, provides machines with the ability to interpret and act upon human language.

For this work, which aims at creating a shared E2E testing benchmark, we focused on test scripts written in standard programming languages that are accessible to all researchers. Proprietary-code CRT tools (e.g., Selenese-based) were excluded, as were pure NLT approaches where only natural language descriptions are used. Consequently, the benchmark includes test suites that are either: (a) developed from scratch using the SBT approach, or (b) produced using a hybrid approach, where an initial CRT recording is manually refined and exported to standard programming languages (Leotta et al., 2022b; Oruç and Ovatman, 2016). To further facilitate studies on NLP-based Web Testing, we provide Gherkin descriptions corresponding to each test script in the benchmark.

### 2.1. Gherkin

Prior to writing the test scripts in a specific approach, it is essential to define the test cases using a specification language. Gherkin[1] is widely used for this purpose (Cerioli et al., 2020; Chandorkar et al., 2022). Specifically, Gherkin is a Domain-Specific Language (DSL) that allows software behavior to be described without specifying implementation details. This makes it ideal for sharing scenario descriptions among teams and for formalizing test cases. The language is organized using keywords such as:

- Feature: provides a high-level description of the test
- Example/Scenario: gives a concrete example
- Given: specifies the initial context
- When: indicates an action performed
- And: indicates an additional action
- Then: describes the expected outcome

Fig. 1 illustrates a simple Gherkin specification for a test case in the MediaWiki test suite part of BEWT. The scenario aims to verify the successful creation of a new user. Such structured natural-language descriptions are particularly useful for NLP-based Web Testing and Generative AI research (see, e.g., (Leotta et al., 2024a,b)).

### 2.2. Script-based web test automation (SBT)

Script-Based Testing involves writing test scripts in programming languages such as Java, Python, or Ruby, using libraries designed to interact with browsers. These libraries provide high-level APIs that allow commands to perform actions on the application, such as clicking buttons, entering data, and submitting forms. Test scripts typically include assertions, implemented via widely used libraries such as xUnit (e.g., JUnit for Java), to verify that expected outcomes occur during execution.

Figs. 2 and 3 show an example of a test script implementing the simple scenario from Fig. 1. Fig. 2 depicts the `BaseTest` class, which contains common methods inherited by all test classes. `@Before` and `@After` annotations define setup and teardown steps, such as initializing the WebDriver, opening the browser, and closing it at the end of the test. Selenium WebDriver[2] (Cerioli et al., 2020) is used to execute the scripts. The code in Fig. 3 follows the *Page Object* design pattern[3,4], which encapsulates page elements and functionalities in dedicated classes. Fig. 4 shows a snippet of the `CreateAccountPage` page object. The Page Object pattern improves maintainability and reduces duplication (Leotta et al., 2016a, 2023), with benefits also during initial test suite development (Leotta et al., 2020).

Each Page Object (PO) represents a web page's elements as objects and exposes methods to perform actions or assertions. This separation of concerns decouples test logic from implementation details, such as the specific locators used to find elements. All element interactions and assertions are centralized in the PO, making them reusable across multiple test cases.

In Fig. 4, web elements are initialized via the Page Factory pattern with `@FindBy` annotations specifying locators. Methods in the `@Test` body (Fig. 3) call PO methods like `setUsername`, while assertions (`assertEquals`) verify expected results.

Programmable test scripts offer flexibility and reusability, supporting loops, conditionals, logging, exceptions, and parameterized test cases. These advantages are enhanced by the Page Object pattern, which improves readability, maintainability, and reduces coupling (Leotta et al., 2016a, 2022a; García et al., 2023). Drawbacks include the need for programming skills, adherence to software development best practices, and significant initial development effort (Leotta et al., 2016a, 2020).

## 3. Motivation

The main goal of this work is to provide a valuable resource for both academic and industrial researchers in the field of E2E Web testing. Specifically, our aim is to deliver a collection of test suites that are immediately usable, facilitating the empirical evaluation of novel testing solutions. Research papers proposing new E2E Web testing approaches, typically implemented via tools, often require experimental validation to demonstrate their effectiveness. However, carrying out such evaluations necessitates access to an adequate number of test suites. Creating these test suites from scratch is generally impractical, as it may

[1] https://cucumber.io/docs/gherkin/reference/

[2] https://www.selenium.dev/documentation/webdriver/

[3] http://martinfowler.com/bliki/PageObject.html

[4] https://code.google.com/p/selenium/wiki/PageObjects

```
package mediawiki;

import org.junit.After;
import org.junit.Before;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;

import org.openqa.selenium.remote.RemoteWebDriver;
import po.MainPage;

import java.net.MalformedURLException;
import java.net.URL;

public class BaseTest {

  public static WebDriver driver;
  public static final String app_url = "http://localhost:8080";

  @Before
  public void setUp() {
    setupNativeBrowser();
    driver.manage().window().maximize();
    driver.get(app_url);
  }

  protected void setupNativeBrowser() {
    ChromeOptions options = new ChromeOptions();
    options.addArguments("--disable-search-engine-choice-screen", "--headless=new", "--disable-gpu",
        "--screen-info={1920x1080}", "--lang=en");
    options.setBrowserVersion("127");
    driver = new ChromeDriver(options);
  }

  protected void setupRemoteWebDriver() {
    ChromeOptions options = new ChromeOptions();
    options.addArguments("--no-sandbox", "--headless=new", "--disable-gpu", "--screen-info={1920x1080}",
        "--lang=en");
    try {
      driver = new RemoteWebDriver(new URL("http://localhost:4444/wd/hub"), options);
    } catch (MalformedURLException e) {
      e.printStackTrace();
    }
  }

  protected MainPage loginAsAdmin() {
    return new MainPage(driver)
        .login()
        .setUsername("admin")
        .setPassword("e2eW3Bt3s71nGB3nchM4rK")
        .login();
  }

  protected MainPage loginAsUser() {
    return new MainPage(driver)
        .login()
        .setUsername("User001")
        .setPassword("e2eW3Bt3s71nGB3nchM4rK")
        .login();
  }

  @After
  public void tearDown() {
    driver.quit();
  }
}
```

**Fig. 2.** Example of test script for MediaWiki: base class.

```
package mediawiki;

import static org.junit.Assert.assertEquals;
import org.junit.Test;
import po.CreateAccountPage;

public class CreateUserTest extends BaseTest {

  @Test
  public void createUser() {
    CreateAccountPage account = loginAsAdmin()
        .specialPages()
        .createAccount()
        .setUsername("User001")
        .setPassword("Password001")
        .confirmPassword("Password001")
        .setRealName("Real Name 001")
        .create();

    assertEquals("The user account for User001 (talk) has been created.", account.getSuccessMessage());
  }
}
```

**Fig. 3.** Example of test script for MediaWiki: code of the test script.

```java
package po;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;

public class CreateAccountPage extends PageObject {

  @FindBy(id = "wpName2")
  protected WebElement username;
  @FindBy(id = "wpPassword2")
  protected WebElement password;
  @FindBy(id = "wpRetype")
  protected WebElement confirmPassword;
  @FindBy(id = "wpRealName")
  protected WebElement realName;
  @FindBy(id = "wpCreateaccount")
  protected WebElement createAccountBtn;
  public CreateAccountPage(WebDriver driver) { super(driver); }

  public CreateAccountPage setUsername(String usr) {
    username.sendKeys(usr); return this;
  }
  public CreateAccountPage setPassword(String psw) {
    password.sendKeys(psw); return this;
  }
  public CreateAccountPage confirmPassword(String psw) {
    confirmPassword.sendKeys(psw); return this;
  }
  public CreateAccountPage setRealName(String rn) {
    realName.sendKeys(rn); return this;
  }
  public CreateAccountPage create() {
    createAccountBtn.click(); return new CreateAccountPage(driver);
  }
  public String getSuccessMessage() {
    return driver.findElement(By.xpath("//*[@id=\"mw-content-text\"]/p[1]")).getText();
  }
  public String getUsernameValidationMessage() {
    return username.getAttribute("validationMessage");
  }
}
```

**Fig. 4.** Example of test script for MediaWiki: snippet from a page object.

introduce bias in the experiments. Additionally, locating open-source test suites associated with existing web applications can be difficult. Beyond academic research, BEWT is also aligned with industrial E2E testing practice, since it relies on widely adopted technologies (Java/Selenium) and coding patterns (e.g., Page Objects) and targets common maintenance and reliability issues.

To our knowledge, there is currently no publicly available benchmark providing a comprehensive set of E2E Web test suites. This issue has been pointed out by several authors (Alian et al., 2025; Meglio et al., 2025), some of whom are developing their own benchmarks at the time of writing. In this work, we introduce our benchmark composed of carefully designed E2E Web test suites, implemented in Java and built on the Selenium WebDriver framework, including multiple versions for some test suites to enable studies on test suite evolution. Java was chosen as the reference programming language due to its wide adoption in both industry and academia and because it is natively supported by many automated testing frameworks, including Selenium. Selenium was selected as the framework because it represents the de facto standard for web automation and GUI testing (García et al., 2024; Leotta et al., 2023; Gojare et al., 2015). It supports a broad range of browsers and platforms and benefits from extensive documentation and a large community. Its robustness and widespread use make it an appropriate choice for benchmarking new E2E testing solutions.

This benchmark has been specifically designed to enable research in the following areas of E2E Web testing:

1. test script generation (e.g., using NLP or generative AI)
2. test dependencies: detection, management and elimination
3. test flakiness reduction
4. test waiting strategies optimization
5. test suite evolution effort reduction
6. test locators robustness improvement

In the remainder of this section, we describe in detail the aforementioned areas of E2E web testing and, for each of them, we provide illustrative empirical study designs (evaluation scenarios) showing how BEWT can be used (gray-boxes, readers may treat these boxes as optional guidance). Importantly, several scenarios are derived from completed empirical studies that we previously published and that were executed on preliminary versions (precursors) of the BEWT test suites; BEWT now consolidates and releases these assets in a standardized, reusable form. Therefore, our goal here is to describe the study designs enabled by BEWT and to point to the corresponding executed studies, rather than to report new results within this paper.

### 3.1. Test generation

Advancements in Natural Language Processing (NLP) and Generative Artificial Intelligence (Gen-AI), particularly in language models like GPT (Generative Pre-trained Transformer), have opened new perspectives for automatic test generation (Yuan et al., 2023; Guilherme and Vincenzi, 2023).

Recently, new tools and frameworks, referred to as code-less and based on NLP (Ricca et al., 2021), have emerged on the market with the goal of reducing development and maintenance costs. The novelty of NLP-based test automation tools/frameworks lies in the fact that test cases are written in natural language, enabling even software testers with limited programming skills to produce executable test scripts. Many vendors have recognized the significant potential of AI in the testing domain and have introduced several NLP-based test automation frameworks/tools (e.g., TestSigma[5], TestRigor[6], and TestProject[7]) capable of interpreting and executing test cases written in natural language. However, the actual benefits of this new category of approaches, particularly regarding cost reduction, have not yet been demonstrated in practice and therefore remain largely speculative.

On the other hand, also approaches based on generative AI like ChatGPT have demonstrated remarkable capabilities in generating coherent

[5] https://testsigma.com/
[6] https://testrigor.com/
[7] https://testproject.io/

and contextually appropriate text across a wide range of, even complex, domains. Indeed, ChatGPT (Open, 2025) has been effectively used in various contexts, including code generation and technical writing. In the context of generating test scripts, ChatGPT offers the ability to produce understandable and adaptable test scripts with minimal or no prior knowledge (Yuan et al., 2023; Guilherme and Vincenzi, 2023). Another example of AI integration into development environments to assist programmers in writing code is GitHub Copilot (GitHub, 2025). Copilot's ability to suggest code based on context provides an opportunity to explore its potential use in generating test scripts.

The detailed Gherkin specification of the test cases included in BEWT can support various kind of studies analyzing the capabilities of these two categories of tools in generating executable test scripts starting from specifications provided as natural language descriptions. An illustrative study design based on the usage of BEWT is provided below.

**Illustrative Study Design (AI-based Test Script Generation)**

This illustrative study design outlines how to evaluate whether AI-based tools can generate executable E2E test scripts from natural-language specifications. The focus is on two tools: (i) ChatGPT and (ii) GitHub Copilot. An empirical study following a similar design was reported in Leotta et al. (2024b), using precursors of the BEWT suites.

**Research Questions.** The study investigates:

- **RQ1:** the syntactic correctness of the generated scripts
- **RQ2:** their executability
- **RQ3:** their functional accuracy with respect to the Gherkin specification
- **RQ4:** the required manual effort to make them executable.

**Subject Systems.** The evaluation can be conducted on the BEWT benchmark, which provides both the precise description of the test cases to be implemented using the Gherkin language (Gherkin) - the starting point for test suite generation - and the actual reference implementation of the corresponding test scripts in the baseline version of the test suite (Base), used to compare the quality of the generated code.

**Procedure.** For each Gherkin test case in the BEWT benchmark, a researcher generates an E2E test script using ChatGPT and GitHub Copilot.
The generated scripts are subsequently executed to record failures, classify their causes, and assess the extent of manual corrections needed to achieve a fully executable version. The process is repeated across all applications to evaluate robustness and generalizability.

**Metrics.** We collect: (i) syntactic correctness rate, (ii) compilation success rate, (iii) execution success rate, (iv) functional correctness with respect to the Gherkin steps, and (v) the amount of manual editing effort (LOC and number of edits). When applicable, we also track the kinds of errors produced (e.g., wrong locators, missing waits, incorrect assertions).

**Summary.** This study design provides a controlled and reproducible setting to assess the effectiveness of AI-based tools in generating executable E2E test scripts from natural-language descriptions. Such a study, when conducted on BEWT, enables comparative studies on the overall effectiveness of AI-assisted testing approaches

### 3.2. Test dependencies

Within an E2E test suite, an order-dependent test script is defined as a script whose result can change depending on the execution sequence of other test scripts run beforehand. These dependencies impose a strict ordering of test execution within the suite, which may restrict the use of techniques such as test selection, prioritization, and parallel execution. Although several methods and tools have been proposed to detect dependencies in test suites, only one approach has been specifically designed, implemented, and validated for E2E Web test suites (Biagiola et al., 2019). Every test suite included in this benchmark contains such dependencies, making them suitable candidates for evaluating tools intended to identify and eliminate test dependencies. An illustrative study design based on the usage of BEWT is provided below.

**Illustrative Study Design (Test Dependencies)**

This sample empirical study design aims to assess the effectiveness and efficiency of a hypothetical tool, named ***DepTool***, designed for detecting test dependencies in E2E testing. The study design leverages the BEWT benchmark as a controlled and diverse experimental environment. An empirical study following a similar design was reported in Biagiola et al. (2019), Olianas et al. (2025), using precursors of the BEWT suites.

**Research Questions.** The study addresses the following research questions:

- **RQ1 (Effectiveness):** How effective is *DepTool* in detecting E2E test dependencies?
- **RQ2 (Performance):** What is the computational overhead introduced by *DepTool*, and what runtime savings does it achieve compared to validating full dependency graphs?

**Subject Systems.** The evaluation can be conducted on the BEWT benchmark. Each application in BEWT provides multiple test suites characterized by different design and synchronization strategies, such as the use of the Page Object Pattern, explicit and implicit waits, and asynchronous interaction handling. This diversity makes BEWT particularly suitable for studying dependency phenomena and assessing how tools like *DepTool* perform across a variety of realistic E2E testing contexts. Since all BEWT test suites contain test dependencies, this evaluation can, in principle, be conducted using any of the test suite variants included in the benchmark. However, test-dependency detection tools typically require deterministic test suite executions to produce reliable results. Given the higher flakiness observed in the explicit-wait versions (Exp-Wait) (see details in Section 4.3), these should be excluded from the study, and the implicit-wait versions should be used instead, for example the baseline one (Base).

**Procedure.** Starting from the baseline (Base) version of the BEWT test suites, the researcher first mitigated test flakiness by refining synchronization and verified stability through multiple runs. A baseline configuration was established by validating dependencies using the original execution order, without filtering or graph optimization. We then applied *DepTool* under multiple configurations, combining different strategies. For each configuration, a validated Test Dependency Graph (TDG) was derived and used to automatically generate parallelizable test schedules that preserved true dependencies among tests.

**Metrics.** The evaluation metrics were defined as follows:

- $\mathbf{M_{RQ1}}$**:** degree of correctness and completeness (precision and recall) of the dependencies detected by *DepTool* compared to a gold standard (e.g., manually computed)
- $\mathbf{M_{RQ2}}$**:** analysis of the time and runtime reduction

**Summary.** This experimental design enables a systematic and reproducible evaluation of test dependency analysis using the BEWT benchmark. By leveraging BEWT's diversity of web applications and test architectures, we can quantitatively assess *DepTool*'s capability to balance accuracy, performance, and parallelization efficiency in realistic E2E testing scenarios.

### 3.3. Test flakiness

Test flakiness represents a challenge in software testing, characterized by test scripts producing inconsistent results when executed on the same version of the system under test. Nevertheless, given the complex nature of flakiness and its potential to arise from multiple concurrent factors, it may occur when the suites are executed under different conditions (e.g., varying web server or database versions, or different browsers).

The benchmark test suites can be employed in studies similar to those by Moran et al. (2020), where tests are executed across different environments-such as diverse browsers, operating systems, screen sizes, or network conditions-to identify scenarios that may trigger flakiness. Additionally, flakiness can be intentionally introduced, for example, by altering the implemented waiting strategies.

Overall, the test suites provided in this benchmark, and in particular the Exp-Wait versions, are designed to support the evaluation of novel techniques and tools aimed at detecting, analyzing, and mitigating test flakiness. An illustrative study design based on the usage of BEWT is provided below.

**Illustrative Study Design (Flakiness Detection)**

This sample empirical study design aims to quantify how much every test suite is affected by flakiness, and what are the possible conditions that may cause it. An empirical study following a similar design was reported in Olianas et al. (2026), using the version of BEWT presented in this paper.

**Research Questions.** The study addresses the following research questions:

- **RQ1 (Prevalence)** How much is each test suite affected by flakiness?
- **RQ2 (Causes)** What are the environmental factors that may increase or reduce the flakiness? With "environmental factor" we mean each aspect that is not strictly related to the test code (like, for example, locators or waiting strategies), but can still affect the results of the execution. Possible environmental factors are the versions of used softwares, the operating system, the use of the headless mode.

**Subject Systems.** Given the higher flakiness observed in the explicit-wait versions (Exp-Wait), these test suites should be used in the study.

**Metrics.** The evaluation metrics can be defined as follows:

- $\mathbf{M_{RQ1}}$**:** number of detected failures over number of test suite runs.
- $\mathbf{M_{RQ2}}$**:** environmental factors under which the highest numbers of failures happened

**Procedure.** In this study design, some environmental factors that may increase or reduce the amount of flakiness in the test suite should be identified, for example:

- the version of every software involved in the test suite execution (Java, Selenium WebDriver, the browser)
- the operating system
- the use of the headless mode
- the specific browser driver being used (standard ChromeDriver for local execution or RemoteWebDriver for execution in containerized or remote browsers).

The test suites must be executed for a sufficient amount of time for each combination of environmental factors. Given the exponential growth of the number of executions that this may cause, it is important that the selected environmental factors are few and meaningful.

**Summary.** This experimental design enables to quantify the amount of flakiness that affect each test suite, and the possible environmental factors that may increase it or reduce it, possibly allowing the identification of the root cause of flakiness in some cases.

### 3.4. Waiting strategies

Waiting strategies in E2E Web testing are mechanisms employed to ensure that page elements are fully loaded before interactions occur. Given the inherently asynchronous nature of E2E Web testing, every action targeting a page element must wait for that element to be available. The Selenium WebDriver framework provides two primary types of waiting strategies: explicit waits and implicit waits. Explicit waits allow waiting for specific, potentially complex conditions on individual elements, whereas implicit waits are applied globally across the entire test script. Additionally, E2E testers often utilize thread sleeps, which are programming-language constructs that pause execution for a defined duration, to ensure page elements are ready. Although thread sleeps have limitations, they remain commonly used, particularly in combination with implicit waits. According to Selenium WebDriver documentation[8], mixing implicit and explicit waits within the same script is discouraged, as it can lead to unpredictable or even infinite wait times (Raghavendra, 2021).

All test suites in this benchmark are designed to use implicit waits, supplemented with thread sleeps when necessary. This setup facilitates comparative studies on the efficiency and effectiveness of different waiting strategies and supports empirical validation of new tools for automating wait management (Olianas et al., 2022). Furthermore, for each web application included in the benchmark, we provide an additional version of the test suite using explicit waits. This allows researchers to conduct comparative studies of waiting strategies from multiple perspectives, such as execution time, occurrence of flakiness, and effort required for test suite evolution. For applications with multiple versions included in the benchmark, the explicit wait-based suite is provided only for the most recent version. An illustrative study design based on the usage of BEWT is provided below.

[8] Selenium WebDriver documentation for waits https://www.selenium.dev/documenta-tion/webdriver/waits/

**Illustrative Study Design (Waiting Strategies)**

This sample empirical study design aims to assess the effectiveness and efficiency of a hypothetical tool, named ***WaitReplacer***, designed to automatically replace static thread sleeps with dynamic synchronization strategies in E2E test suites. The study follows established guidelines for empirical research in software engineering and is grounded on the BEWT benchmark to ensure replicability and experimental diversity. An empirical study following a similar design was reported in Olianas et al. (2022), using precursors of the BEWT suites.

**Research Questions.** The evaluation is driven by the following research questions:

- **RQ1 (Replacement Effectiveness):** How effective is *WaitReplacer* at automatically replacing thread sleeps with explicit waits?
- **RQ2 (Performance):** What is the computational cost of applying *WaitReplacer* to large-scale test suites?
- **RQ3 (Human Effort Reduction):** How much manual effort can be saved by using *WaitReplacer* instead of performing replacements manually?
- **RQ4 (Runtime Impact):** How does synchronization replacement affect the total execution time of BEWT test suites?

**Subject Systems.** The study is conducted on a selection of test suites from the BEWT benchmark, which includes 36 Selenium WebDriver-based E2E test suites for 8 web applications of varying size and complexity. Since this study design requires the presence of thread sleeps in the original test suites, the study can be conducted with all the test suites that contain thread sleeps: baseline (Base), baseline evolution (Base+), no page objects (No-POs) and full XPaths (Full-XPath) are all suitable for the experiment. The only kind of test suite that is not suitable is explicit wait (Exp-Wait), since there are only two thread sleeps among the 8 test suites. Together, these systems encompass almost 1000 automated test methods and more than seven hundred instances of static sleeps (`Thread.sleep`) across diverse architectures, including Page Object-based and direct interaction test designs. This diversity makes BEWT an ideal experimental ground to observe synchronization-related behaviors and assess the generalizability of the results.

**Procedure.** Each BEWT test suite is provided as input to *WaitReplacer*, which analyzed the test code to locate `Thread.sleep` invocations and replace them with explicit synchronization constructs. The tool operates according to a set of predefined transformation rules, ensuring the correctness and stability of the resulting tests. To validate the replacements, the modified test suites can be run multiple times observing potential non-deterministic failures. All systems were deployed using BEWT's Docker-based setup, which ensures isolation, reproducibility, and efficient management of test dependencies and state restoration.

**Metrics.** We measured the following metrics to answer the research questions:

- $\mathbf{M_{RQ1}}$**:** proportion of `Thread.sleep` statements correctly replaced by explicit waits.
- $\mathbf{M_{RQ2}}$**:** execution time of the replacement process for each test suite.
- $\mathbf{M_{RQ3}}$**:** estimated reduction in manual effort, computed by comparing the time required for manual replacement to the automated execution time of *WaitReplacer*.
- $\mathbf{M_{RQ4}}$**:** variation in total test suite runtime before and after applying *WaitReplacer*, capturing potential speed-up from optimized synchronization.

**Summary.** This experimental design enables a systematic evaluation of automated synchronization management in E2E testing. By leveraging the controlled environment and application diversity of the BEWT benchmark, the study quantitatively assesses *WaitReplacer*'s ability to improve test reliability, reduce maintenance effort, and enhance execution performance across realistic web testing scenarios.

### 3.5. Test suite evolution effort reduction

The main benefits of adopting test automation include the ability to: (1) execute test scripts more frequently, (2) detect defects at earlier stages of development, and (3) reuse test code across subsequent releases of the web application for regression testing. However, a major challenge associated with test automation is the fragility of test scripts. A fragile test is one that fails when minor modifications occur in the web application under test. Specifically, as the web application evolves to accommodate requirement changes, bug fixes, or new functionalities, test scripts may fail. For example, they might not find certain links, input fields, or submission buttons. Fixing these test scripts can become labor-intensive and costly, requiring manual effort from software testers. Automatic evolution of test suites is therefore an active area of research. Depending on the type of maintenance applied to the web application, software testers must carry out a series of repair actions on the test scripts, which can be classified into two categories: logical and structural.

- Logical Changes refer to modifications in the functionality of the web application. To repair test scripts impacted by such changes, testers must adjust one or more steps within the broken test scripts. For example, a possible change request (CR1) requiring a logical repair could involve enhancing security through stronger authentication. This might include adding a new field to the login form (in addition to the standard username and password), such as an OTP code.
- Structural Changes involve modifications limited to the layout or structure of the web page. For instance, on an authentication page, the text on the login button might change to "Submit" (CR2), or the identifier of a web element might be updated (e.g., from id="UID" to id="UserID") (CR3). Typically, the impact of a structural change is smaller than that of a logical change. Repairing test scripts after structural changes often only requires modifying one or more locator lines that reference the changed elements.

The test suites provided in this benchmark enable the evaluation of approaches aimed at automatically repairing broken test scripts. Indeed, for half of the web applications, we included two test suites corresponding to different application versions. These serve as reference test suites, effectively providing a ground truth for how the test suites should appear after evolution. An illustrative study design based on the usage of BEWT is provided below.

**Illustrative Study Design (Automated Test Suite Evolution)**

This sample empirical study design aims to assess the effectiveness and efficacy of an hypothetical tool, named ***AutoEvolver***, that aims to automatically evolve a test suite for a newer version of the application under test. The tool takes as input a test suite and the version application under test for which the test suite should be evolved. AutoEvolver executes the test suite against the new version, and automatically repairs every broken test.

**Research Questions.** The study addresses the following research questions:

- **RQ1 (Effectiveness):** How effective is *AutoEvolver* in correctly evolving test suites?
- **RQ2 (Code Quality):** What is the code quality of the test suites evolved by *AutoEvolver* with respect to manually evolved test suites?

**Subject Systems.** This evaluation can be conducted using all the baseline (Base) versions of the test suites that also provide a baseline evolution (Base+) version. The baseline version can be used as input for *AutoEvolver*, the evolution version can be used as a reference to compare the test suites generated by *AutoEvolver* to answer RQ2.**Metrics.** The evaluation metrics can be defined as follows:

- $\mathbf{M_{RQ1}}$**:** number of tests that pass without failures in the evolved test suites over total.
- $\mathbf{M_{RQ2}}$**:** mutation score and code coverage of the automatically evolved test suites compared to the corresponding baseline evolution (Base+) test suites in BEWT

**Procedure.** Each baseline version can be provided as input to *AutoEvolver*. To answer RQ1, the researcher can simply count how many test scripts have been successfully evolved (i.e. test scripts that pass with no failures on the newer version of the application under test) and how many test scripts report failures. To answer RQ2, the researcher should fix some metrics for code quality (e.g. robustness of locators, number of code smells, lines of code) and compare the metrics of the test suites evolved by *AutoEvolver* with the metrics of the BEWT baseline evolution corresponding test suites.

**Summary.** This experimental design enables the empirical evaluation of the *AutoEvolver* tool: the baseline versions provide the experimental object, while the baseline evolution versions can serve as a reference to compare the code quality of automatically evolved test suites with manually evolved ones.

### 3.6. Test locators robustness improvement

Locators are structured identifiers used by test automation tools to accurately find and interact with web elements on the GUI. As previously noted, locators are employed to identify and fill input sections of a web page (such as form fields), navigate through pages (e.g., locating and clicking links), or validate output (for instance, identifying web elements that display computational results).

Research has highlighted the challenges and costs associated with maintaining *locators* during the evolution of the web application under test (Leotta et al., 2014; Hammoudi et al., 2016). Leotta et al. (2014) demonstrated that even small changes in the application can significantly affect locators. Consequently, locators are critical for addressing the *fragility problem* in automated test suites. When a web application changes due to evolving requirements existing automated test scripts may fail, requiring manual correction by testers. This repair process is costly, and even minor changes, such as layout adjustments, can render test scripts *fragile*. Using robust locators helps mitigate this fragility. Research on generating robust locators has been active for many years (Nass et al., 2023; Leotta et al., 2021, 2016b, 2015); while this task falls under test suite maintenance, it warrants a separate discussion.

The test suites included in this benchmark facilitate the evaluation of techniques aimed at producing robust locators. By providing two versions of each applicable test suite, a mapping between locators is inherently available. Each web element present in both versions is identified with a locator, e.g., *l*1 in the first version and *l*2 in the second version. A tool or method that successfully generates robust locators should be able to locate the same element associated with *l*2 in the updated version of the application, using only the information from the first version. An illustrative study design based on the usage of BEWT is provided below.

**Illustrative Study Design (Locators Robustness)**

This sample empirical study design aims to assess the robustness and performance of **RobustGen**, a hypothetical novel algorithm for generating resilient XPath locators in web test automation. An empirical study following a similar design was reported in Leotta et al. (2016b), using precursors of the BEWT suites (in particular the locators contained in the test suites).

**Research Questions.** The evaluation focuses on the following research questions:

- **RQ1 (Comparison with the State of the Art):** Does *RobustGen* generate XPath locators that are more robust than those produced by existing state-of-practice tools and research algorithms?
- **RQ2 (Structural Factors):** How do structural characteristics of the generated XPath locators (e.g., depth, attributes, predicates) influence their robustness?
- **RQ3 (Performance):** Is the runtime overhead of *RobustGen* acceptable for practical web testing scenarios?

**Subject Systems.** The experiment can be conducted on the BEWT benchmark. BEWT provides multiple releases for some systems, enabling the study of locator robustness across evolving web interfaces. The large number of web applications included in BEWT allows RobustGen to be tested on realistic and heterogeneous DOM structures. The benchmark's extensive use of XPath-based locators, Page Object models, and synchronization mechanisms makes it an ideal testbed for evaluating locator generation algorithms under evolution. In particular, the applications that provide a baseline evolution (Base+) version are particularly suitable for this kind of experiment, since the evolution version can serve as an oracle that provides functioning locators for a subsequent version of the application under test. In these cases the starting point for generating the robust locators can be the elements selected by the XPaths provided in the (Full-XPath) versions or the ones selected by the various kind of locators in the baseline (Base) versions.

**Procedure.** For each web page in BEWT's applications, the relevant web elements involved in test interactions or assertions are identified using the locators and tracked their corresponding counterparts across application versions. *RobustGen* was then applied to the first release of each web page to automatically generate XPath locators. For comparison, we also collected locators produced by representative state-of-practice and research tools, including Selenium IDE, Similo (Nass et al., 2023), and Sidereal (Leotta et al., 2021). Each locator was tested on the subsequent version of the same BEWT application to determine whether it still correctly identified the intended element (a locator is considered "broken" when it fails to select the same element). All executions can be performed within BEWT's Docker-based infrastructure, ensuring reproducibility and environmental control.

**Metrics.** The following metrics are collected to address the research questions:

- $\mathbf{M_{RQ1}}$**:** number and proportion of broken locators per algorithm.
- $\mathbf{M_{RQ2}}$**:** correlation between locator robustness and features such as depth, number of predicates, and use of text or attribute constraints.
- $\mathbf{M_{RQ3}}$**:** average execution time per locator generation, compared across all algorithms.

**Summary.** By leveraging the diverse and versioned applications of the BEWT benchmark, this experiment provides a rigorous assessment of different locators generation tools and algorithms under realistic evolution scenarios. Such a study would allow direct comparison of *RobustGen* against established tools and algorithms, quantifying both the robustness gains and computational trade-offs of the proposed approach. Through BEWT's reproducible setup and controlled experimental design, *RobustGen*'s contribution to reliable, evolution-resilient locator generation can be effectively validated and benchmarked.

## 4. Procedure

To create the benchmark, we followed a structured process. First, we collected several existing test suites developed within prior E2E testing research works (Leotta et al., 2024a; Biagiola et al., 2019; Olianas et al., 2025, 2022; Leotta et al., 2018). Next, we carried out a ***test suite selection*** process to identify the most suitable test suites for inclusion in the benchmark. Finally, we ***enhanced the selected test suites*** to guarantee uniformity in their code structure and adherence to key best practices in E2E web testing (e.g., using the Page Object (PO) pattern, Page Factory pattern, PO method chaining, and implicit wait strategy). The following sections provide a detailed description of these stages.

### 4.1. Test suites selection

From the initially gathered test suites, we selected those that met the following criteria:

- The application under test must be sufficiently complex to support at least 20 non-trivial test cases. Consequently, applications such as Spring Petclinic[9] and Addressbook[10], commonly used in E2E testing research, were excluded due to their simplicity, making them inadequate as examples of modern web applications.
- The application under test must have at least one version released after 2015. This ensures that outdated, unmaintained applications are excluded, focusing the benchmark on contemporary web applications.
- The application under test must be open source to allow redistribution among researchers without licensing restrictions. Open-source access also enables full code inspection, which is essential for white-box testing approaches.
- The application under test must be easily deployable in one or more Docker containers, minimizing installation and configuration overhead for researchers using the benchmark.
- For half of the selected test suites, a second version of the application must be available, significantly different from the original version. This allows test suites to vary meaningfully across versions, enabling studies on test suite evolution by comparing different releases of the same application (we plan to include this for all applications in future benchmark releases).

[9] https://spring-petclinic.github.io/
[10] https://sourceforge.net/projects/php-addressbook/

### 4.2. Test suites versions derivation

The test suites we collected were originally developed for various studies with distinct objectives, leading to differences in features, structure, and code organization. To maximize standardization, we applied the Page Object (PO) pattern to all test suites, along with the Page Factory pattern where applicable. All page objects implement method chaining, meaning that each method of a page object returns the page object corresponding to the web application page to which the invoked method navigates.

We also standardized the waiting strategies by adopting the *implicit wait* strategy across all test suites, supplemented with thread sleeps where necessary.

For each application, we defined the Gherkin specification corresponding to a version of the base test suite.

Finally, we evolved four out of the eight test suites (Joomla, MantisBT, PrestaShop, and Bludit) to provide two versions of the test suite for each application.

### 4.3. Test suites flakiness reduction

The quality of the test suite code has been improved. All test suites were thoroughly checked for flakiness, and every issue that could reasonably be identified was addressed.

To quantify the presence of test flakiness, we executed each test suite at least 50 times. We selected this number because, in our experience, it provides a reasonable compromise between flakiness-detection capability and execution time. The number of repetitions used in the literature varies significantly across studies and often depends on the nature and execution cost of the tests under analysis (Palomba and Zaidman, 2019). A single E2E web test script typically requires an execution time ranging from several seconds to over a minute for more complex scenarios. Consequently, running test suites containing up to 40 test scripts (as in BEWT) for 50 iterations results in 2000 individual test-script executions per test-suite version, which corresponds to several hours of total execution time.

In our analysis, we classify a test suite as having *low flakiness* if no more than one test script fails across 50 test-suite re-executions. This corresponds to at most one failure out of 2000 individual test-script executions (for a test suite composed of 40 test scripts), i.e., a per-test flakiness rate below 0.05%. Interpreted at the level of test-suite runs, this also corresponds to at most one flaky outcome over 50 executions (i.e., below 2% suite-level flakiness). We consider both thresholds to indicate a low flakiness level for the purposes of this study.

The results show variability depending on the type of test suite: the *baseline* versions were the most stable, with every baseline and baseline evolution test suite completing all runs without any errors. The *no page object* versions also demonstrated high stability, with no errors detected in all runs for any test suite. Some flakiness remains in the *explicit wait* versions: in detail, during the runs we observed failures in Bludit, Expresscart, Joomla, Kanboard, Mediawiki, and Prestashop. The number of failures per 50 test suite executions ranged from 1 to 11. It is important to note that flakiness can be heavily influenced by software versions (particularly the browser) and the execution environment (operating system, CPU, RAM, etc.), so users may observe failure rates that differ significantly from our results.

### 4.4. Test suites publish and documentation

We published the test suites, together with the accompanying documentation, on GitHub starting from the following URL: https://sepl.dibris.unige.it/BEWTExtended.php. For each application, we provide installation instructions and a recommended Docker container. For applications that require additional setup steps after container deployment, we include an automated installer that executes these steps automatically. These installers are implemented as E2E test scripts using

**Table 1**
Summary of the features of all test suite types.

| Test suite type | Page Object pattern | Waiting strategy | Locator type | Flakiness |
|---|---|---|---|---|
| *Baseline (Base)* | yes | implicit waits + thread sleeps | many | low |
| *Baseline evolution (Base+)* | yes | implicit waits + thread sleeps | many | low |
| *Explicit Wait (Exp-Wait)* | yes | explicit waits | many | medium |
| *No page objects (No-PO)* | no | implicit waits + thread sleeps | many | low |
| *Full XPaths (Full-XPath)* | no | implicit waits + thread sleeps | absolute XPaths only | low |

Selenium WebDriver, interacting with the applications via their installation wizards.

## 5. Test suites

For each application, we provide at least four versions of the test suites: **baseline** (Base), **explicit wait** (Exp-Wait), a version that uses explicit waits as waiting strategies, **no page objects** (No-PO), a version that does not employ the Page Object pattern, and **full XPaths** (Full-XPath), a version that uses only absolute XPaths as locators. Moreover, for four out of the eight applications, we also include a **baseline evolution** (Base+) version, an evolution of the (Base) version for a subsequent release of the application under test. This section presents a detailed description of each test suite type, while Table 1 summarizes their main characteristics. In the Table low flakiness indicates no more than one test script failure across 50 test suite executions, while medium flakiness indicates fewer than 15 failures across the same number of runs. Note that 50 test suite executions correspond to running between 1150 test scripts (Bludit: 23 test scripts * 50) and 2000 test scripts (Claroline: 40 test scripts * 50).

### 5.1. Common features

All the test suites are implemented in Java, use Selenium WebDriver to interact with web pages, and rely on JUnit 4.12 as the testing framework. They are organized as Maven projects to simplify dependency management, and each test suite follows a one-test-script-per-class structure. All test suites can be executed either on a local browser (installed on the machine or retrieved via Selenium Manager) or on a Selenium Grid; the BaseTest classes provide two corresponding execution modes, one using a standard ChromeDriver and one using a RemoteWebDriver.

For some applications under test, additional installation steps are required beyond deploying the Docker container. In these cases, the associated test suites include an automated installer (implemented as a WebDriver test) that performs the required steps automatically, thereby minimizing manual intervention.

To support research on test flakiness, all test suites include two auxiliary scripts-a Windows Batch script and a Linux Bash script-for repeated executions. These scripts set up the application under test, run the test suite, store the results, tear down the application, and then proceed with the next iteration.

### 5.2. Baseline

The **baseline** version is, for each web application, the original test suite from which all other variants are derived. All baseline test suites adopt the Page Object pattern and, where applicable, the Page Factory pattern. The Page Object pattern can be implemented at different levels of abstraction, ranging from simply encapsulating calls to *driver.findElement(…)* to designing a full Domain-Specific Language (DSL). In this work, we implement page objects using method chaining (i.e., methods in page objects return the page object corresponding to the page reached after the executed action). Test suites are organized with one test per class. This choice was made to facilitate execution and editing of single test scripts, maintaining the code cleaner.

To reduce code duplication, we adopt inheritance: all test classes extend a *BaseTest* class (shown in Fig. 2), which defines the setup and teardown methods, as well as the *WebDriver* instances and global variables (e.g., authentication credentials).

Regarding waiting strategies, all baseline versions rely on implicit waits, supplemented by thread sleeps when required.

Table 2 provides a summary of the key characteristics of the baseline test suites.

### 5.3. Baseline evolution

The **baseline evolution** version of the test suites is available for four applications: Bludit, Joomla, MantisBT, and Prestashop. This version is an adaptation of the original test suite to ensure compatibility with a subsequent release of the application under test. We selected application versions that exhibited significant differences from the original ones. To determine whether two versions were sufficiently distinct, we executed the original test suite on the newer version and proceeded with the evolution only when a substantial number of tests failed.

The evolved test suites preserve all structural characteristics of the original ones (Page Object model, use of implicit waits), introducing only the modifications required to ensure compatibility with the newer application version (typically updates to locators or different action sequences needed to accomplish certain tasks). In one case (Prestashop), the evolved test suite contains one fewer test than the baseline version, as the corresponding functionality is not present in the selected version.

These test suites can be used in studies focused on test suite evolution (i.e., maintaining a test suite across different versions of the application under test). Table 3 summarizes the characteristics of the evolved test suites.

### 5.4. Explicit wait

The **explicit wait** version of the test suites is an adaptation of the baseline version (or the baseline evolution version, for the four web applications that include that variant) that replaces implicit waits with explicit waits. To develop these test suites, we removed the implicit wait configuration from the baseline (or baseline evolution) version, executed the test suite, and introduced explicit waits wherever failures occurred.

To improve code readability and maintainability, we encapsulated all explicit wait invocations in a dedicated *Wait* class, as shown in Fig. 5. Figs. 6 and 7 show the same page object method from the *baseline evolution* and *explicit wait* versions of the Prestashop test suite, respectively.

In the explicit wait version, the final action (a click) in the page object method must be protected by an *elementToBeClickable* explicit wait (encapsulated by the *waitClickability* method) to ensure its successful execution.

These test suites can serve as useful resources for studies investigating waiting strategies and flakiness in E2E web testing. Table 4 summarizes the characteristics of the explicit wait test suites. All suites include explicit waits, with substantial variability in their number: test suites that show low flakiness or few failures when executed without waiting mechanisms require fewer explicit waits, whereas less stable suites require a larger number of them.

**Table 2**
Summary of *baseline* (Base) test suites features.

| App | Version | Test Cases | Page Objects | Locators | Thread Sleeps | Explicit waits | Total LOC | PO LOC | Tests LOC |
|---|---|---|---|---|---|---|---|---|---|
| *Bludit* | 2.3.4 | 23 | 16 | 77 | 8 | 0 | 1167 | 634 | 533 |
| *Claroline* | 1.11.10 | 40 | 22 | 130 | 3 | 0 | 1860 | 1015 | 845 |
| *Expresscart* | 1.19 | 31 | 25 | 105 | 22 | 0 | 1630 | 883 | 698 |
| *Joomla* | 3.10.11 | 35 | 29 | 137 | 7 | 0 | 1946 | 1016 | 856 |
| *Kanboard* | 1.2.15 | 30 | 26 | 102 | 11 | 0 | 1450 | 838 | 612 |
| *MantisBT* | 1.2.0 | 37 | 19 | 107 | 14 | 0 | 1690 | 812 | 844 |
| *Mediawiki* | 1.40.0 | 27 | 15 | 83 | 31 | 0 | 1461 | 759 | 629 |
| *Prestashop* | 1.6.1.23 | 35 | 31 | 132 | 10 | 0 | 1927 | 1110 | 751 |

**Table 3**
Summary of *baseline evolution* (Base + ) test suites features.

| App | Version | Test Cases | Page Objects | Locators | Thread Sleeps | Explicit waits | Total LOC | PO LOC | Tests LOC |
|---|---|---|---|---|---|---|---|---|---|
| *Bludit* | 3.13.1 | 25 | 21 | 87 | 15 | 0 | 1378 | 778 | 600 |
| *Joomla* | 4.2.0 | 35 | 30 | 140 | 12 | 0 | 1866 | 1059 | 787 |
| *MantisBT* | 2.25.4 | 37 | 19 | 105 | 11 | 0 | 1711 | 773 | 904 |
| *Prestashop* | 1.7.8.5 | 34 | 30 | 157 | 14 | 0 | 1981 | 1184 | 731 |

```
public class Wait {

  protected WebDriverWait wait;

  public Wait(WebDriver driver) {
    wait = new WebDriverWait(driver, 30);
  }

  public void waitVisibility(WebElement element) {
    wait.until(ExpectedConditions.visibilityOf(element));
  }

  public void waitVisibility(By locator) {
    wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
  }

  public void waitClickability(WebElement element) {
    wait.until(ExpectedConditions.elementToBeClickable(element));
  }

  public void waitClickability(By locator) {
    wait.until(ExpectedConditions.elementToBeClickable(locator));
  }

  public void waitForTextToBe(By locator, String text) {
    wait.until(ExpectedConditions.textToBe(locator, text));
  }

  public void waitForTextToBeContained(By locator, String text) {
    wait.until(ExpectedConditions.textToBePresentInElementLocated(locator, text));
  }

  public boolean waitForTextToDisappear(By locator, String text) {
    return wait.until(ExpectedConditions.invisibilityOfElementWithText(locator, text));
  }

  public void waitClickabilityRefreshed(By locator) {
    wait.until(ExpectedConditions.refreshed(ExpectedConditions.elementToBeClickable(locator)));
  }

  public void waitStaleness(WebElement element) {
    wait.until(ExpectedConditions.stalenessOf(element));
  }
}
```

**Fig. 5.** Class encapsulating invocations of explicit waits.

### 5.5. No page objects

The **no page objects** version of the test suites is an adaptation of the baseline version (or the baseline evolution version, for the four web applications that include that variant) that does not use the Page Object model. To build this version, we inlined the code from the page object methods directly into the test script bodies, applying the necessary adjustments (e.g., replacing Page Factory constructs with explicit *driver.findElement* calls).

Fig. 8 shows the same test script as in the previous examples, but taken from the *no page objects* version.

These test suites can support studies that analyze or evaluate the Page Object model, including techniques and tools for automatically generating page objects.

Table 5 summarizes the characteristics of the no page objects test suites. In some suites, the Total LOC metric differs from the Tests LOC metric even though no page object code is present; this discrepancy is due to additional utility code (e.g., installers, *JavaScriptExecutors*) located outside the test package.

### 5.6. Full XPaths

Finally, the **full XPaths** version of the test suites is an adaptation of the no page objects versions that use only absolute XPaths as locators. This version was originally developed to support the empirical evaluation of TestQuest (Clerissi et al., 2025), a gamification tool designed to improve locator robustness. Thus full XPaths version does not employ

```
public SupplierPage removeSupplier() {
  removeDropDown.click();
  driver.findElement(By.xpath("//*[@id=\"supplier_grid_table\"]/tbody/tr[1]/td[7]/div/div/div/a[2]"))
          .click();
  driver.findElement(By.xpath("//*[@id=\"supplier-grid-confirm-modal\"]/div/div/div[3]/button[2]"))
          .click();
  return new SupplierPage(driver);
}
```

**Fig. 6.** Page object method from the *baseline evolution* version of the Prestashop test suite.

```
public SupplierPage removeSupplier() {
  removeDropDown.click();
  driver.findElement(By.xpath("//*[@id=\"supplier_grid_table\"]/tbody/tr[1]/td[7]/div/div/div/a[2]"))
          .click();
  By locator = By.xpath("//*[@id=\"supplier-grid-confirm-modal\"]/div/div/div[3]/button[2]");
  wait.waitClickability(locator);
  driver.findElement(locator).click();
  return new SupplierPage(driver);
}
```

**Fig. 7.** The same page object method, but from the *explicit wait* version of the Prestashop test suite.

**Table 4**
Summary of *explicit wait* (Exp-Wait) test suites features.

| App | Version | Test Cases | Page Objects | Locators | Thread Sleeps | Explicit waits | Total LOC | PO LOC | Tests LOC |
|---|---|---|---|---|---|---|---|---|---|
| *Bludit* | 3.13.1 | 25 | 21 | 87 | 0 | 101 | 1480 | 836 | 609 |
| *Claroline* | 1.11.10 | 40 | 22 | 130 | 0 | 139 | 2071 | 1158 | 870 |
| *Expresscart* | 1.19 | 31 | 25 | 105 | 1 | 91 | 1719 | 865 | 711 |
| *Joomla* | 4.2.0 | 35 | 30 | 140 | 0 | 141 | 2006 | 1152 | 791 |
| *Kanboard* | 1.2.15 | 30 | 26 | 102 | 0 | 108 | 1569 | 908 | 618 |
| *MantisBT* | 2.25.4 | 37 | 19 | 105 | 0 | 115 | 1841 | 832 | 907 |
| *Mediawiki* | 1.40.0 | 27 | 15 | 85 | 1 | 52 | 1395 | 678 | 668 |
| *Prestashop* | 1.7.8.5 | 34 | 30 | 157 | 0 | 68 | 2079 | 1262 | 754 |

```
public class RemoveSupplier extends BaseTest {

  @Test
  public void test_remove_supplier(){
    driver.findElement(By.name("email")).sendKeys(adminMail);
    driver.findElement(By.name("passwd")).sendKeys(adminPsw);
    driver.findElement(By.name("submitLogin")).click();
    driver.findElement(By.xpath("//*[@id=\"subtab-AdminCatalog\"]/a")).click();
    driver.findElement(By.id("subtab-AdminParentManufacturers")).click();
    driver.findElement(By.id("subtab-AdminSuppliers")).click();
    driver.findElement(By.xpath("//*[@id=\"supplier_grid_table\"]/tbody/tr[1]/td[7]/div/div/a[2]"))
            .click();
    driver.findElement(By.xpath("//*[@id=\"supplier_grid_table\"]/tbody/tr[1]/td[7]/div/div/div/a[2]"))
            .click();
    driver.findElement(By.xpath("//*[@id=\"supplier-grid-confirm-modal\"]/div/div/div[3]/button[2]"))
            .click();

    assertTrue(driver.findElement(By.className("alert-text")).getText()
          .contains("Successful deletion"));
  }
}
```

**Fig. 8.** Test script from Prestashop, *no page objects* version.

the Page Object pattern and relies on implicit waits, supplemented with thread sleeps where necessary.

Due to certain limitations of the tested tool, this version has a slightly different code organization compared to the No-PO (no page objects) version: in the full XPaths version, each web element is first assigned to a variable before being used, rather than being accessed directly via *driver.findElement*.

This test suite can be useful for studies on locator robustness and test code refactoring. Fig. 9 shows the same test script as in the previous examples, but from the full XPaths version of the Prestashop test suite. Table 6 summarizes the characteristics of the full XPaths test suites.

### 5.7. Summary

Table 7 provides an overview of the characteristics of our test suites. The test suites vary in size, ranging from 23 to 40 test scripts and from 585 to 2079 lines of code (LOC). All test suites (except for the explicit wait versions) rely on Selenium WebDriver's implicit wait to manage asynchronous calls. Nevertheless, in some cases, thread sleeps are still required to ensure stability.

In total, 30 test suites contain at least one thread sleep, while 6 test suites do not. Among the test suites with thread sleeps, the number of sleeps ranges from 1 to 89. The explicit wait versions include between 52 and 141 explicit waits. The page object versions of the test suites contain between 15 and 31 page objects.

Across all test suites, a total of 1166 test scripts were developed, covering a diverse set of scenarios for evaluating End-to-End (E2E) web testing techniques. The test suites incorporate 492 Page Objects, following the Page Object Pattern to improve maintainability and reusability.

Regarding interaction with web elements, the test suites include 8284 locators. To handle asynchronous behavior and reduce flakiness, 718 instances of thread sleeps were introduced where necessary, com-

**Table 5**
Summary of *no page objects* (No-PO) test suites features.

| App | Version | Test Cases | Page Objects | Locators | Thread Sleeps | Explicit waits | Total LOC | PO LOC | Tests LOC |
|---|---|---|---|---|---|---|---|---|---|
| *Bludit* | 3.13.1 | 25 | 0 | 247 | 27 | 0 | 585 | 0 | 585 |
| *Claroline* | 1.11.10 | 40 | 0 | 710 | 20 | 0 | 1262 | 0 | 1243 |
| *Expresscart* | 1.19 | 31 | 0 | 256 | 38 | 0 | 757 | 0 | 708 |
| *Joomla* | 4.2.0 | 35 | 0 | 528 | 19 | 0 | 1224 | 0 | 1204 |
| *Kanboard* | 1.2.15 | 30 | 0 | 303 | 12 | 0 | 667 | 0 | 667 |
| *MantisBT* | 2.25.4 | 37 | 0 | 348 | 32 | 0 | 1036 | 0 | 980 |
| *Mediawiki* | 1.40.0 | 27 | 0 | 363 | 89 | 0 | 901 | 0 | 901 |
| *Prestashop* | 1.7.8.5 | 34 | 0 | 349 | 33 | 0 | 930 | 0 | 863 |

```
public class RemoveSupplierTest extends BaseTest {

   @Test
   public void test_remove_supplier(){
      WebElement emailField = driver.findElement(
          By.xpath("/html/body/div/div/div/div[3]/div/div[1]/form/div[1]/input"));
      emailField.sendKeys(adminMail);
      WebElement pswField = driver.findElement(
          By.xpath("/html/body/div/div/div/div[3]/div/div[1]/form/div[2]/input"));
      pswField.sendKeys(adminPsw);
      WebElement loginBtn = driver.findElement(
          By.xpath("/html/body/div/div/div/div[3]/div/div[1]/form/div[3]/button"));
      loginBtn.click();
      WebElement catalog = driver.findElement(By.xpath("/html/body/nav/div/ul/li[4]/a"));
      catalog.click();
      WebElement brandsAndSuppliers = driver.findElement(
          By.xpath("/html/body/nav/div/ul/li[4]/ul/li[5]/a"));
      brandsAndSuppliers.click();
      WebElement suppliersTab = driver.findElement(By.xpath("/html/body/div[1]/div[2]/ul/li[2]/a"));
      suppliersTab.click();
      WebElement threePointsIcon = driver.findElement(
          By.xpath("/html/body/div[2]/div/div/div/div[3]/div/div/div[2]/div/div/div[2]/div/form/table
                    /tbody/tr[3]/td[7]/div/div/a[2]"));
      threePointsIcon.click();
      WebElement delete = driver.findElement(
          By.xpath("/html/body/div[2]/div/div/div/div[3]/div/div/div[2]/div/div/div[2]/div/form/table
                    /tbody/tr[3]/td[7]/div/div/div/a[2]"));
      delete.click();
      WebElement confirmDelete = driver.findElement(
          By.xpath("/html/body/div[9]/div/div/div[3]/button[2]"));
      confirmDelete.click();
      WebElement successMsg = driver.findElement(By.xpath("/html/body/div[2]/div/div/div/div[2]/div"));

      assertTrue(successMsg.getText().contains("Successful deletion"));
   }
}
```

**Fig. 9.** Test script from Prestashop, *full XPath* version.

**Table 6**
Summary of *full XPath (Full-XPath)* test suites features.

| App | Version | Test Cases | Page Objects | Locators | Thread Sleeps | Explicit waits | Total LOC | PO LOC | Tests LOC |
|---|---|---|---|---|---|---|---|---|---|
| *Bludit* | 3.13.1 | 25 | 0 | 238 | 34 | 0 | 839 | 0 | 839 |
| *Claroline* | 1.11.10 | 40 | 0 | 540 | 20 | 0 | 1780 | 0 | 1780 |
| *Expresscart* | 1.19 | 31 | 0 | 253 | 38 | 0 | 975 | 0 | 975 |
| *Joomla* | 4.2.0 | 35 | 0 | 477 | 15 | 0 | 1595 | 0 | 1595 |
| *Kanboard* | 1.2.15 | 30 | 0 | 296 | 11 | 0 | 984 | 0 | 984 |
| *MantisBT* | 2.25.4 | 37 | 0 | 489 | 77 | 0 | 1742 | 0 | 1686 |
| *Mediawiki* | 1.40.0 | 27 | 0 | 280 | 73 | 0 | 1154 | 0 | 1081 |
| *Prestashop* | 1.7.8.5 | 34 | 0 | 334 | 20 | 0 | 1180 | 0 | 1114 |

plementing the waiting strategies. Additionally, 815 explicit waits were implemented to enhance test reliability by synchronizing interactions with dynamic elements.

Overall, the test suites consist of 51,838 lines of code (LOC). This extensive dataset provides a valuable benchmark for researchers aiming to evaluate and compare different approaches in E2E web testing, addressing challenges such as test flakiness, test suite evolution, and automated testing strategies.

## 6. Related works

In this section, we summarize existing research relevant to this benchmark, with a particular focus on collections of E2E tests. We discuss related work after presenting BEWT's construction and characteristics, so that comparisons can be made with full context on the benchmark's scope and design choices. One of the most well-known and widely used datasets for software testing is the Software-artifact Infrastructure Repository (SIR) (Do et al., 2005), a research repository that provides a large collection of software artifacts, including source code of real programs, test suites, test inputs, and expected outputs. Unlike our work, SIR is a broad repository containing various types of software artifacts and is not specifically focused on E2E testing of web applications.

Soto-Sánchez et al. (2020) released a dataset containing six regression bugs injected into three web applications, along with the corresponding tests designed to detect these bugs. In contrast to our work, the applications in their dataset are not real-world web applications but are purpose-built examples created solely for their study. Furthermore,

**Table 7**
Summary of test suites' features.

| App | Version | Variant | Test Cases | Page Objects | Locators | Thread Sleeps | Explicit waits | Total LOC | PO LOC | Tests LOC |
|---|---|---|---|---|---|---|---|---|---|---|
| *Bludit* | 2.3.4 | Base | 23 | 16 | 77 | 8 | 0 | 1167 | 634 | 533 |
| *Bludit* | 3.13.1 | Base+ | 25 | 21 | 87 | 15 | 0 | 1378 | 778 | 600 |
| *Bludit* | 3.13.1 | Exp-Wait | 25 | 21 | 87 | 0 | 101 | 1480 | 836 | 609 |
| *Bludit* | 3.13.1 | No-PO | 25 | 0 | 247 | 27 | 0 | 585 | 0 | 585 |
| *Bludit* | 3.13.1 | Full-XPath | 25 | 0 | 238 | 34 | 0 | 839 | 0 | 839 |
| *Claroline* | 1.11.10 | Base | 40 | 22 | 130 | 3 | 0 | 1860 | 1015 | 845 |
| *Claroline* | 1.11.10 | Exp-Wait | 40 | 22 | 130 | 0 | 139 | 2071 | 1158 | 870 |
| *Claroline* | 1.11.10 | No-PO | 40 | 0 | 710 | 20 | 0 | 1262 | 0 | 1243 |
| *Claroline* | 1.11.10 | Full-XPath | 40 | 0 | 540 | 20 | 0 | 1780 | 0 | 1780 |
| *Expresscart* | 1.19 | Base | 31 | 25 | 105 | 22 | 0 | 1630 | 883 | 698 |
| *Expresscart* | 1.19 | Exp-Wait | 31 | 25 | 105 | 1 | 91 | 1719 | 865 | 711 |
| *Expresscart* | 1.19 | No-PO | 31 | 0 | 256 | 38 | 0 | 757 | 0 | 708 |
| *Expresscart* | 1.19 | Full-XPath | 31 | 0 | 253 | 38 | 0 | 975 | 0 | 975 |
| *Joomla* | 3.10.11 | Base | 35 | 29 | 137 | 7 | 0 | 1946 | 1016 | 856 |
| *Joomla* | 4.2.0 | Base+ | 35 | 30 | 140 | 12 | 0 | 1866 | 1059 | 787 |
| *Joomla* | 4.2.0 | Exp-Wait | 35 | 30 | 140 | 0 | 141 | 2006 | 1152 | 791 |
| *Joomla* | 4.2.0 | No-PO | 35 | 0 | 528 | 19 | 0 | 1224 | 0 | 1204 |
| *Joomla* | 4.2.0 | Full-XPath | 35 | 0 | 477 | 15 | 0 | 1595 | 0 | 1595 |
| *Kanboard* | 1.2.15 | Base | 30 | 26 | 102 | 11 | 0 | 1450 | 838 | 612 |
| *Kanboard* | 1.2.15 | Exp-Wait | 30 | 26 | 102 | 0 | 108 | 1569 | 908 | 618 |
| *Kanboard* | 1.2.15 | No-PO | 30 | 0 | 303 | 12 | 0 | 667 | 0 | 667 |
| *Kanboard* | 1.2.15 | Full-XPath | 30 | 0 | 296 | 11 | 0 | 984 | 0 | 984 |
| *MantisBT* | 1.2.0 | Base | 37 | 19 | 107 | 14 | 0 | 1690 | 812 | 844 |
| *MantisBT* | 2.25.4 | Base+ | 37 | 19 | 105 | 11 | 0 | 1711 | 773 | 904 |
| *MantisBT* | 2.25.4 | Exp-Wait | 37 | 19 | 105 | 0 | 115 | 1841 | 832 | 907 |
| *MantisBT* | 2.25.4 | No-PO | 37 | 0 | 348 | 32 | 0 | 1036 | 0 | 980 |
| *MantisBT* | 2.25.4 | Full-XPath | 37 | 0 | 489 | 77 | 0 | 1742 | 0 | 1686 |
| *Mediawiki* | 1.40.0 | Base | 27 | 15 | 83 | 31 | 0 | 1461 | 759 | 629 |
| *Mediawiki* | 1.40.0 | Exp-Wait | 27 | 15 | 85 | 1 | 52 | 1395 | 678 | 668 |
| *Mediawiki* | 1.40.0 | No-PO | 27 | 0 | 363 | 89 | 0 | 901 | 0 | 901 |
| *Mediawiki* | 1.40.0 | Full-XPath | 27 | 0 | 280 | 73 | 0 | 1154 | 0 | 1081 |
| *Prestashop* | 1.6.1.23 | Base | 35 | 31 | 132 | 10 | 0 | 1927 | 1110 | 751 |
| *Prestashop* | 1.7.8.5 | Base+ | 34 | 30 | 157 | 14 | 0 | 1981 | 1184 | 731 |
| *Prestashop* | 1.7.8.5 | Exp-Wait | 34 | 30 | 157 | 0 | 68 | 2079 | 1262 | 754 |
| *Prestashop* | 1.7.8.5 | No-PO | 34 | 0 | 349 | 33 | 0 | 930 | 0 | 863 |
| *Prestashop* | 1.7.8.5 | Full-XPath | 34 | 0 | 334 | 20 | 0 | 1180 | 0 | 1114 |
| *TOTAL* | | | 1166 | 492 | 8284 | 718 | 815 | 51838 | 18552 | 31923 |
| *MIN* | | | 23 | 15 | 77 | 1 | 52 | 585 | 634 | 533 |
| *MAX* | | | 40 | 31 | 710 | 89 | 141 | 2079 | 1262 | 1780 |

the provided tests do not follow any specific design pattern (e.g., the Page Object pattern), and each application includes only one to three tests - just enough to detect the injected regressions.

Gortázar et al. (2021) proposed a methodology for collecting E2E tests from GitHub projects based on 12 criteria related to project folder and file naming. While their methodology is sound, it does not include verification steps to filter out toy projects or demo applications (Meglio et al., 2025). We did not adopt their approach because the goal of our work is not to collect existing E2E tests, but rather to provide a curated dataset of carefully designed E2E tests that incorporate different design patterns and waiting strategies, intended for research studies on E2E testing.

Fuad and Sakib (2023) proposed WebEV, a dataset consisting of E2E web tests from open-source GitHub projects. Unlike our dataset, their focus is on Cypress-based test scripts, while our work focuses on Selenium WebDriver scripts. Selenium has long been considered the de facto standard for browser automation, although more recently alternative tools such as Cypress, Puppeteer, and Playwright have emerged. Therefore, both Selenium and Cypress remain relevant and interesting choices (García et al., 2024). Moreover, at the time of writing, their dataset is not publicly available.

In 2019, Gyimesi et al. proposed BugJS (Gyimesi et al., 2019), a benchmark designed to support research on JavaScript analysis, testing, debugging, and automated program repair by providing a curated set of 453 real, reproducible JavaScript bugs. Their benchmark and ours can be considered complementary: while we provide test suites that run on correctly functioning systems (free of known bugs), they provide a curated set of reproducible JavaScript bugs, which is particularly useful for assessing the fault-detection capabilities of test suites. Similar works include Defects4J (Just et al., 2014) (a collection of 357 Java bugs from real-world programs) and Bugs.jar (Saha et al., 2018) (a collection of 1158 bugs and patches from eight open-source Java projects).

Finally, in 2025, Di Meglio et al. introduced E2EGit (Meglio et al., 2025), a dataset that contains 43,670 GUI web tests extracted from 472 GitHub repositories, along with 271 performance tests collected from 84 repositories. The performance tests rely on tools such as JMeter and Locust, while the GUI tests are built using Selenium, Playwright, Cypress, and Puppeteer, and are written in multiple programming languages, including Java, JavaScript, TypeScript, and Python. The repositories were collected using the SEART GitHub Search Engine[11], ensuring the exclusion of toy projects and including only relevant open-source web applications. To the best of our knowledge, this is currently the most comprehensive dataset on E2E web testing. Although both E2EGit and our work focus on datasets for E2E web testing, they differ significantly. E2EGit is a large-scale dataset of E2E tests collected from real-world applications, covering multiple programming languages and testing technologies. Because the tests in E2EGit originate from various applications developed by different authors, the quality of the code, the structure and size of the test suites, the design patterns used, and the waiting strategies can vary considerably. In contrast, our dataset, although smaller, consists of test suites that follow a uniform coding style and directory structure. This consistency makes BEWT suitable for *controlled benchmarking* that are difficult to support with large-scale mined datasets

[11] https://seart-ghs.si.usi.ch/

such as E2EGit. Because BEWT provides systematically derived variants from the same base suites, it supports analyses where a single factor (e.g., Page Objects, synchronization strategy, locator style) is changed while keeping the test intent and application constant. This also enables tool evaluations that require paired reference suites (e.g., assessing Page Object generation or explicit-wait insertion against a curated counterpart), and evolution/maintenance studies with a ground-truth reference thanks to the availability of aligned suites across releases (Base/Base + ). Moreover these test suite versions can also serve as valuable examples for university courses or professional training programs on E2E web testing.

## 7. Conclusion and future work

In this work, we addressed the need for a standardized benchmark in E2E web testing research by introducing a comprehensive collection of test suites. Our benchmark, BEWT, comprises 36 Java Selenium WebDriver-based test suites spanning 8 different web applications, each crafted to facilitate the evaluation of E2E testing techniques. By incorporating variations such as the Page Object Pattern, different waiting strategies, and support for multiple application versions, BEWT provides a controlled and reproducible environment for systematically assessing the effectiveness of novel approaches and tools across diverse applications. By enabling rigorous evaluation, the benchmark contributes to the safe and dependable operation of web systems, reducing the risk of undesired states and potential failures. Overall, BEWT establishes a solid foundation for the development and assessment of automated tools and strategies aimed at enhancing the reliability, maintainability, and general dependability of web applications, directly supporting software dependability objectives in critical and complex web systems. In addition to research, BEWT can serve as teaching material for university courses and professional training on E2E web testing.

Our benchmark comprises 1166 test scripts distributed across multiple test suites, incorporating 492 Page Objects and 8284 locators to facilitate research in E2E web testing. To handle asynchronous behavior, the test suites include 718 thread sleeps and 815 explicit waits.

To enhance usability and reproducibility, we have packaged all test suites with Docker containers, ensuring seamless deployment. With a total of 51,838 lines of code (LOC), this dataset is intended to support researchers in developing and comparing new approaches to E2E testing, reducing biases in experimental validation and fostering more reliable assessments of emerging techniques.

Future work will focus on expanding the benchmark with additional applications and diversifying test suite characteristics to further improve its applicability. In particular, we plan to introduce new applications, new programming languages (e.g., Python, JavaScript) and new testing frameworks (like Cypress, Playwright or Puppeteer) to expand the applicability of our benchmark. We also aim to explore automated techniques for maintaining and evolving test suites across multiple software versions. By continuing to refine and extend this benchmark, we hope to contribute to more rigorous and effective research in web application testing. We do not yet report direct evidence of industrial adoption; collecting adoption feedback and conducting industry-facing evaluations is part of our future work.

## CRediT authorship contribution statement

**Dario Olianas:** Writing – review & editing, Writing – original draft, Validation, Software, Resources, Project administration, Methodology, Investigation, Data curation, Conceptualization; **Maurizio Leotta:** Writing – review & editing, Writing – original draft, Validation, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Data curation, Conceptualization; **Filippo Ricca:** Writing – review & editing, Writing – original draft, Validation, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Data curation, Conceptualization.

## Data availability

BEWT benchmark dataset can be found on GitHub (link in the paper)

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgements

This study was partially carried out within the “EndGame - Improving End-to-End Testing of Web and Mobile Apps through Gamification” project (2022PCCMLF) - Next Generation EU within the PRIN 2022 program (D.D.104 - 02/02/2022 Ministero dell’Università e della Ricerca). This manuscript reflects only the authors’ views and opinions and the Ministry cannot be considered responsible for them.

## References

Alian, P., Nashid, N., Shahbandeh, M., Shabani, T., Mesbah, A., 2025. Feature-driven end-to-end test generation. In: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE). IEEE Computer Society, pp. 678.

Berner, S., Weber, R., Keller, R.K., 2005. Observations and lessons learned from automated testing. In: Proceedings of the 27th International Conference on Software Engineering. Association for Computing Machinery, New York, NY, USA, p. 571–579, ICSE’ 05. https://doi.org/10.1145/1062455.1062556

Biagiola, M., Stocco, A., Mesbah, A., Ricca, F., Tonella, P., 2019. Web test dependency detection. In: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ACM, p. 154–164, ESEC/FSE 2019. https://doi.org/10.1145/3338906.3338948

Cerioli, M., Leotta, M., Ricca, F., 2020. What 5 million job advertisements tell us about testing: a preliminary empirical investigation. In: Proceedings of 35th ACM/SIGAPP Symposium on Applied Computing (SAC 2020). ACM, pp. 1586–1594. https://doi.org/10.1145/3341105.3373961

Chandorkar, A., Patkar, N., Di Sorbo, A., Nierstrasz, O., 2022. An exploratory study on the usage of gherkin features in open-source projects. In: 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER). IEEE, pp. 1159–1166.

Clerissi, D., Olianas, D., Leotta, M., Ricca, F., 2025. Improving Locators Quality for E2E Testing through Gamification with TestQuest. In: Proceedings of 18th International Conference on the Quality of Information and Communications Technology (QUATIC 2025). Springer,

Do, H., Elbaum, S., Rothermel, G., 2005. Supporting controlled experimentation with testing techniques: an infrastructure and its potential impact. Empir. Softw. Eng. 10 (4), 405–435.

Fuad, M. M.N., Sakib, K., 2023. WebEV: a dataset on the behavior of testers for web application end to end testing. In: 2023 IEEE/ACM 31st International Conference on Program Comprehension (ICPC). IEEE, pp. 79–83.

García, B., Alamo, J.M., Leotta, M., Ricca, F., 2024. Exploring browser automation: a comparative study of selenium, cypress, puppeteer, and playwright. In: Bertolino, A., Faria, J.P., Lago, P., Semini, L. (Eds.), Proceedings of 17th International Conference on the Quality of Information and Communications Technology (QUATIC 2024). Springer. 2178, pp. 142–149, CCIS. https://doi.org/10.1007/978-3-031-70245-7_10

García, B., Ricca, F., del Alamo, J.M., Leotta, M., 2023. Enhancing web applications observability through instrumented automated browsers. J. Syst. Softw. 203, 111723. https://doi.org/10.1016/j.jss.2023.111723

GitHub, I., 2025. Github copilot. Accessed: 01 November 2025. https://copilot.github.com/.

Gojare, S., Joshi, R., Gaigaware, D., 2015. Analysis and design of selenium webdriver automation testing framework. Proced. Comput. Sci. 50, 341–346.

Gortázar, F., Maes-Bermejo, M., Gallego, M., Contreras Padilla, J., 2021. Looking for the needle in the haystack: end-to-end tests in open source projects. In: Paiva, A. C.R., Cavalli, A.R., Ventura Martins, P., Pérez-Castillo, R. (Eds.), Quality of Information and Communications Technology. Springer International Publishing, Cham, pp. 40–48.

Guilherme, V., Vincenzi, A., 2023. An initial investigation of chatGPT unit test generation capability. In: Proceedings of the 8th Brazilian Symposium on Systematic and Automated Software Testing. ACM, p. 15–24, SAST’ 23.

Gyimesi, P., Vancsics, B., Stocco, A., Mazinanian, D., Beszédes, A., Ferenc, R., Mesbah, A., 2019. BugsJS: a benchmark of javascript bugs. In: 2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST), pp. 90–101. https://doi.org/10.1109/ICST.2019.00019

Hammoudi, M., Rothermel, G., Tonella, P., 2016. Why do record/replay tests of web applications break? In: 2016 IEEE International Conference on Software Testing, Verification and Validation (ICST), pp. 180–190. https://doi.org/10.1109/ICST.2016.16

Just, R., Jalali, D., Ernst, M.D., 2014. Defects4j: a database of existing faults to enable controlled testing studies for java programs. In: Proceedings of the 2014 International Symposium on Software Testing and Analysis, pp. 437–440.

Leotta, M., Biagiola, M., Ricca, F., Ceccato, M., Tonella, P., 2020. A family of experiments to assess the impact of page object pattern in web test suite development. In: Proceedings of 13th IEEE International Conference on Software Testing, Verification and Validation (ICST 2020). IEEE, pp. 263–273. https://doi.org/10.1109/ICST46399.2020.00035

Leotta, M., Clerissi, D., Ricca, F., Tonella, P., 2013. Capture-Replay vs. programmable web testing: an empirical assessment during test case evolution. In: Proceedings of 20th Working Conference on Reverse Engineering (WCRE 2013). IEEE, pp. 272–281. https://doi.org/10.1109/WCRE.2013.6671302

Leotta, M., Clerissi, D., Ricca, F., Tonella, P., 2014. Visual vs. DOM-based web locators: an empirical study. In: Sven Casteleyn, Gustavo Rossi, M.W. (Ed.), Proceedings of 14th International Conference on Web Engineering (ICWE 2014). Springer. 8541, pp. 322–340, LNCS. https://doi.org/10.1007/978-3-319-08245-5_19

Leotta, M., Clerissi, D., Ricca, F., Tonella, P., 2016a. Approaches and tools for automated end-to-end web testing. Adv. Comput. 101, 193–237. https://doi.org/10.1016/bs.adcom.2015.11.007

Leotta, M., García, B., Ricca, F., 2022a. An empirical study to quantify the setup and maintenance benefits of adopting WebDriverManager. In: Vallecillo, A., Visser, J., Pérez-Castillo, R. (Eds.), Proceedings of 15th International Conference on the Quality of Information and Communications Technology (QUATIC 2022). Springer. 1621, pp. 31–45, CCIS. https://doi.org/10.1007/978-3-031-14179-9_3

Leotta, M., García, B., Ricca, F., Whitehead, J., 2023. Challenges of end-to-end testing with selenium WebDriver and how to face them: a survey. In: Proceedings of 16th IEEE International Conference on Software Testing, Verification and Validation (ICST 2023). IEEE, pp. 339–350. https://doi.org/10.1109/ICST57152.2023.00039

Leotta, M., Molinari, A., Ricca, F., 2022b. ASSESSOR: A PO-based WebDriver test suites generator from selenium IDE recordings. In: Proceedings of 15th International Conference on Software Testing, Verification and Validation (ICST 2022). IEEE, pp. 389–399. https://doi.org/10.1109/ICST53961.2022.00045

Leotta, M., Ricca, F., Marchetto, A., Olianas, D., 2024a. An empirical study to compare three web test automation approaches: NLP-based, programmable, and capture & replay. J. Softw.: Evolut. Process (JSEP) 36 (5), e2606. https://doi.org/10.1002/smr.2606

Leotta, M., Ricca, F., Tonella, P., 2021. SIDEREAL: Statistical adaptive generation of robust locators for web testing. J. Softw.: Test. Verificat. Reliab. (STVR) 31. https://doi.org/10.1002/stvr.1767

Leotta, M., Stocco, A., Ricca, F., Tonella, P., 2015. Using multi-locators to increase the robustness of web test cases. In: Proceedings of 8th IEEE International Conference on Software Testing, Verification and Validation (ICST 2015). IEEE, pp. 1–10. https://doi.org/10.1109/ICST.2015.7102611

Leotta, M., Stocco, A., Ricca, F., Tonella, P., 2016b. ROBULA+: An algorithm for generating robust XPath locators for web testing. J. Softw.: Evolut. Process (JSEP) 28 (3), 177–204. https://doi.org/10.1002/smr.1771

Leotta, M., Stocco, A., Ricca, F., Tonella, P., 2018. PESTO: Automated migration of DOM-based web tests towards the visual approach. J. Softw.: Test. Verificat. Reliab. (STVR) 28 (4), e1665. https://doi.org/10.1002/stvr.1665

Leotta, M., Yousaf, H.Z., Ricca, F., Garcia, B., 2024b. Ai-generated test scripts for web E2E testing with ChatGPT and copilot: a preliminary study. In: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering. Association for Computing Machinery, New York, NY, USA, p. 339–344, EASE'24. https://doi.org/10.1145/3661167.3661192

Meglio, S.D., Starace, L. L.L., Pontillo, V., Opdebeeck, R., Roover, C.D., Martino, S.D., 2025. E2EGit: a dataset of end-to-end web tests in open source projects. In: 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR). IEEE Computer Society, Los Alamitos, CA, USA, pp. 836–840. https://doi.org/10.1109/MSR66628.2025.00121

Moran, J., Augusto, C., Bertolino, A., De La Riva, C., Tuya, J., 2020. Flakyloc: flakiness localization for reliable test suites in web applications. J. Web Eng. 19 (2), 267–296.

Nass, M., Alégroth, E., Feldt, R., Leotta, M., Ricca, F., 2023. Similarity-based web element localization for robust test automation. ACM Transact. Softw. Eng. Methodol. (TOSEM) 32 (3). https://doi.org/10.1145/3571855

Olianas, D., Leotta, M., Ricca, F., 2022. SleepReplacer: a novel tool-based approach for replacing thread sleeps in selenium webdriver test code. Softw. Qual. J. (SQJ) 30, 1089–1121. https://doi.org/10.1007/s11219-022-09596-z

Olianas, D., Leotta, M., Ricca, F., 2026. BEWT: A benchmark for end-to-end web testing. In: Taibi, D., Smite, D. (Eds.), Software Engineering and Advanced Applications. Springer Nature Switzerland, Cham, pp. 296–314. https://doi.org/10.1007/978-3-032-04207-1_20

Olianas, D., Leotta, M., Ricca, F., Biagiola, M., Tonella, P., 2025. STILE: A tool for optimizing E2E web test scripts parallelization. J. Syst. Softw. (JSS) 222, 112304. https://doi.org/10.1016/j.jss.2024.112304

Olianas, D., Leotta, M., Ricca, F., Garcia, B., 2026. What makes web end-to-End tests flaky? An empirical study. Automat. Softw. Eng. (ASE), (under review), https://sepl.dibris.unige.it/E2E-Flakiness-Exp.php.

Olianas, D., Leotta, M., Ricca, F., Villa, L., 2021. Reducing flakiness in end-to-end test suites: an experience report. In: Paiva, A. C.R., Cavalli, A.R., Ventura Martins, P., Pérez-Castillo, R. (Eds.), Proceedings of 14th International Conference on the Quality of Information and Communications Technology (QUATIC 2021). Springer. 1439, pp. 3–17, CCIS. https://doi.org/10.1007/978-3-030-85347-1_1

Open, A.I., 2025. ChatGPT.
Accessed: 01 November 2025. https://chat.openai.com/.

Oruç, A.F., Ovatman, T., 2016. Testing of web services using behavior-driven development. In: CLOSER (2), pp. 85–92.

Palomba, F., Zaidman, A., 2019. The smell of fear: on the relation between test smells and flaky tests. Empirical Softw. Engg. 24 (5), 2907–2946. https://doi.org/10.1007/s10664-019-09683-z

Raghavendra, S., 2021. Waits. Apress, Berkeley, CA. pp. 129–142. https://doi.org/10.1007/978-1-4842-6249-8_10

Ricca, F., Marchetto, A., Stocco, A., 2021. Ai-based test automation: a grey literature analysis. In: IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), pp. 263–270. https://doi.org/10.1109/ICSTW52544.2021.00051

Saha, R.K., Lyu, Y., Lam, W., Yoshida, H., Prasad, M.R., 2018. Bugs. jar: a large-scale, diverse dataset of real-world java bugs. In: Proceedings of the 15th International Conference on Mining Software Repositories, pp. 10–13.

Soto-Sánchez, Ó., Maes-Bermejo, M., Gallego, M., Gortázar, F., 2020. A dataset of regressions in web applications detected by end-to-end tests. In: Quality of Information and Communications Technology: 13th International Conference, QUATIC 2020, Faro, Portugal, Springer, pp. 439–448, September 9–11, 2020, Proceedings 13.

Stocco, A., Leotta, M., Ricca, F., Tonella, P., 2017. APOGEN: Automatic page object generator for web testing. Softw. Qual. J. (SQJ) 25 (3), 1007–1039. https://doi.org/10.1007/s11219-016-9331-9

Yuan, Z., Lou, Y., Liu, M., Ding, S., Wang, K., Chen, Y., Peng, X., 2023. No more manual tests? Evaluating and improving chatGPT for unit test generation. arXiv:2305.04207.